

Phase 8: Data Management & Deployment

 **Goal:** Manage financial transaction data efficiently and move project configurations between environments.

1. Data Import Wizard:

Purpose: Import small demo datasets into Salesforce for testing.

Steps Followed:

- Created a sample CSV with ~50 **demo financial transactions** (Transaction ID, Customer ID, Amount, Location, Risk Score, Fraud Flag).
- Used **Data Import Wizard** (Setup → Data Import Wizard → Custom Objects → Transaction).
- Imported the records successfully (Claim/Transaction ID auto-generated if using Auto Number).

Why Important?

Provides test data for fraud monitoring flows, validations, and dashboards.

 **SETUP**
Bulk Data Load Jobs

Monitor Bulk Data Load Jobs [Help for this Page](#)

Monitor the status of recent bulk data load jobs. These jobs are created by Data Loader and other Bulk API client applications.

Quota

Your organization has processed 0 batches in the last 24 hours. Your organization can process 15,000 batches in a 24-hour period.

Resource used in the last 24 hours:
CPU: 0 milliseconds
IO: 0 bytes
Disk: 0 bytes

In Progress

Job ID	Submitted By	Start Time	Status	Job Type	Operation	Object	Records Processed	Records Failed	Progress
No records to display.									

Completed last 7 days

Job ID	Submitted By	Start Time	End Time	Status	Job Type	Operation	Object	Records Processed	Records Failed	Time to Complete (hh:mm:ss)
750gk000000DhvpX	Deshmukh_Sanjivani	9/24/2025, 5:56 AM	9/24/2025, 5:56 AM	Closed	Bulk V1	Insert	Insurance Claim	50	0	00:01

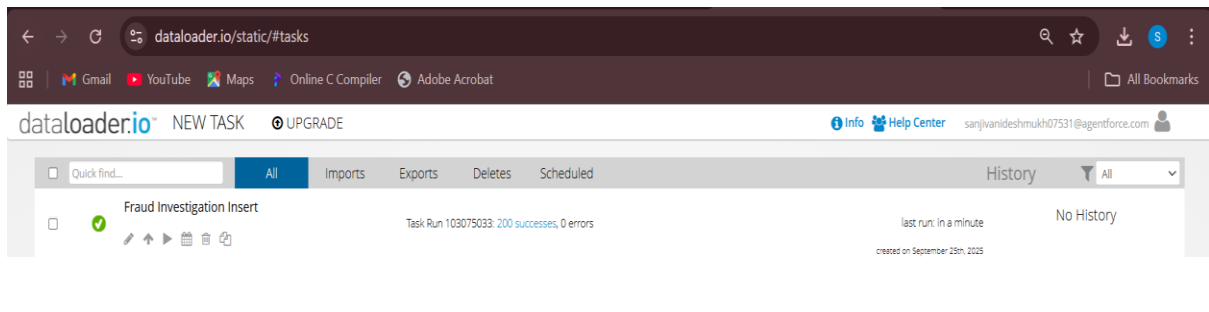
2. Data Loader (Bulk Data):

Purpose: Import or export large datasets (50k+ records).

- Can be used if you want to test system performance with **bulk financial transactions**.
- Supports insert, update, upsert, delete.

Why Important?

Useful for **stress testing fraud detection algorithms** with bigger data.



3. Duplicate Rules:

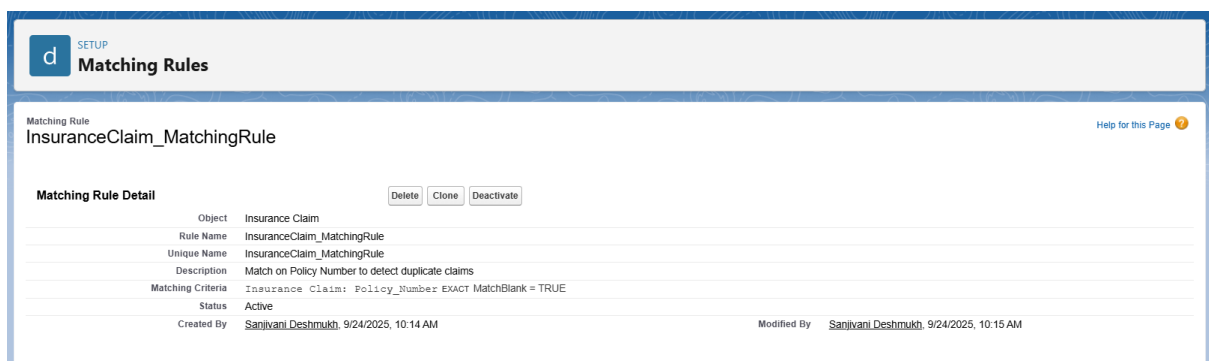
Purpose: Prevent duplicate **transaction records** or **customer profiles**.

Steps:

- Setup → Duplicate Rules → New Rule.
- For **Customer Profile**, match on Email/Phone.
- For **Transactions**, match on Transaction ID.

Why Important?

Fraud detection depends on accurate data — duplicates could create false alerts.



SETUP

Duplicate Rules

Insurance Claim Duplicate Rule

InsuranceClaim_DuplicateRule

Help for this Page

Duplicate Rule Detail

Edit

Delete

Clone

Deactivate

Rule Name	InsuranceClaim_DuplicateRule	Order	1 of 1	Reorder
Description	Prevents duplicate Insurance Claims based on Policy Number.			
Object	Insurance Claim			
Record-Level Security	Enforce sharing rules			
Action On Create	Allow	Operations On Create	☒ Alert	☐ Report
Action On Edit	Allow	Operations On Edit	☒ Alert	☐ Report
Alert Text	Use one of these records?			
Active	☒			
Matching Rule	☒ InsuranceClaim_MatchingRule	☒ Mapped	Matching Criteria	
Conditions	Insurance Claim: Policy_Number EXACT MatchBlank = TRUE			
Created By	Sanjivani Deshmukh 9/24/2025, 10:04 AM		Modified By	
			Sanjivani Deshmukh 9/24/2025, 10:16 AM	

Edit

Delete

Clone

Deactivate

4. Data Export & Backup:

Purpose: Ensure regular backup of fraud data.

Steps:

- Setup → Data Export → Schedule Weekly Export.
- Export all custom objects: Transaction, Customer Profile, Fraud Investigation.
- Store CSV backups securely.

Why Important?

Prevents data loss during errors or deployment, ensures audit history of fraud investigations.

SETUP

Data Export

Monthly Export Service

Help for this Page

Data Export lets you prepare a copy of all your data in salesforce.com. From this page you can start the export process manually or schedule it to run automatically. When an export is ready for download you will receive an email containing a link that allows you to download the file(s). The export files are also available on this page for 48 hours, after which time they are deleted.

Next scheduled export: 10/1/2025, 12:00 AM

Export Now

Schedule Export

5. Change Sets:

Purpose:

Change Sets are normally used to move custom objects, fields, Apex classes, flows, and Lightning pages from Sandbox to Production without re-building.

Steps (real-world process):

1. In Sandbox Org → Setup → Outbound Change Sets → New Change Set.
2. Add components: Custom Objects, Apex Classes, Flows, Validation Rules, Approval Processes, Lightning Pages.
3. Upload Change Set to Target Org.
4. In Production Org → Inbound Change Sets → Deploy.

Note for Our Project:

Since we are working in a Developer Edition Org, Change Sets are not available.

Instead, we documented the process for completeness but used VS Code & SFDX to manage deployments in our project.

6. Unmanaged vs Managed Packages (Optional)

- **Unmanaged:** Share project with classmates → editable.
- **Managed:** Locked → AppExchange style.

Our Case: Use Unmanaged Package if sharing project. No AppExchange publishing needed.

7. ANT Migration Tool:

Purpose: Command-line deployment tool.

- Used in enterprise teams with CI/CD pipelines.
 - Not required for this project.
-

8. VS Code & SFDX:

Purpose:

VS Code with Salesforce CLI (SFDX) gives a developer-friendly way to manage Salesforce metadata, automate deployments, and maintain version control with Git. Unlike Change Sets, this approach works in Developer Orgs and is preferred for real projects.

Steps Followed:**1. Install Salesforce CLI (SFDX):**

- Download from Salesforce CLI website.
- Install it on your system.

2. Install VS Code & Salesforce Extension Pack:

- Install Visual Studio Code.
- In VS Code → Extensions Marketplace → Search Salesforce Extension Pack → Install.

3. Authorize Dev Org in VS Code:

- Open Command Palette in VS Code (Ctrl + Shift + P).
- Type: SFDX: Authorize an Org.
- Choose “Dev Hub” or “Sandbox/Developer Org” → Log in via browser → Allow access.

4. Create/Connect Project:

- In VS Code → Command Palette → SFDX: Create Project with Manifest.
- Select a folder for your project.
- This generates a manifest/package.xml file.

5. Pull Metadata from Org:

- In the terminal (inside your VS Code project folder), run:

`sf project retrieve start --manifest manifest/package.xml`

- This fetches all metadata defined in package.xml (e.g., custom objects, Apex classes, LWCs).

The screenshot shows a VS Code window with a terminal running the command `sf project retrieve start --manifest manifest/package.xml`. The output indicates a successful retrieval of metadata from the Salesforce org. A table titled "Retrieved Source" lists the retrieved items.

```
PS C:\Users\91899\OneDrive\Desktop\Git clone\FraudDetectionLWC> sf project retrieve start --manifest manifest/package.xml
>>
» Warning: @salesforce/cli update available from 2.104.6 to 2.107.6.

- [4/4] Done 0ms

Status: Succeeded

Retrieving Metadata

Retrieving v64.0 metadata from sanjivanideshmukh07531@agentforce.com using the v64.0 SOAP API
✓ Preparing retrieve request 52ms
✓ Sending request to org 608ms
✓ Waiting for the org to respond 2.42s
✓ Done 0ms

Status: Succeeded
Elapsed Time: 3.03s
```

State	Name	Type	Path
Created	ClaimRiskRecalcBatch	ApexClass	force-app\main\default\classes\ClaimRiskRecalcBatch.cls
Created	ClaimRiskRecalcBatch	ApexClass	force-app\main\default\classes\ClaimRiskRecalcBatch.cls-meta.xml
Created	ClaimRiskRecalcScheduler	ApexClass	force-app\main\default\classes\ClaimRiskRecalcScheduler.cls
Created	ClaimRiskRecalcScheduler	ApexClass	force-app\main\default\classes\ClaimRiskRecalcScheduler.cls-meta.xml

Ln 11, Col 60 Spaces: 4 UTF-8 LF Apex Finish Setup